

The University of Nottingham
Malaysia Campus

SCHOOL OF COMPUTER SCIENCE

A LEVEL 2 MODULE, AUTUMN SEMESTER 2016-2017

OPERATING SYSTEMS AND CONCURRENCY

Time allowed TWO Hours

*Candidates must **NOT** start writing their answers until told to do so*

Answer THREE questions out of FOUR

Only silent, self-contained calculators with a single line display are permitted in this examination.

Dictionaries are not allowed with one exception. Those whose first language is not English may use a standard translation dictionary to translate between that language and English provided that neither language is the subject of this examination. Subject specific translation dictionaries are not permitted.

No electronic devices capable of storing and retrieving text, including electronic dictionaries, may be used.

DO NOT turn examination paper over until instructed to do so

1. Memory Management and File Management (25 Marks)

- (a) A process has 4 page frames allocated to it. The following table Table 1.0 lists the page frames, their load time, their last referenced time, the referenced (R) and modified (M) bits. (All numbers are in decimal).

Page frame	Load Time	Last reference	R bit	M bit
0	167	374	1	1
1	321	321	0	0
2	254	306	0	1
3	154	331	1	0

Table 1.0

Which page frame will have its contents replaced for each of the following memory management policies? Explain why in each case.

- (i) FIFO
 - (ii) LRU
 - (iii) NRU [6 Marks]
- (b) (i) Briefly describe the concept of demand paging. [4 Marks]
- (ii) What is *thrashing*? Explain how a system detect and eliminate it? [5 Marks]
- (c) The designer of a new operating system to be used especially for real-time application has proposed the use of virtual memory, so that the system can handle programs too large to fit in the limited memory space sometimes provided on real-time systems. What are the implications of this decision in terms of the way that virtual memory works? [4 Marks]
- (d) Consider a file that has just grown beyond its present space on disk. Describe what steps will be taken next for the following file system implementations?
- (i) A contiguous file. [2 Marks]
 - (ii) A linked non-contiguous file. [2 Marks]
 - (iii) An indexed file. [2 Marks]

2. Process Scheduling and Device Management (25 Marks)

- (a) Consider the ready list containing the processes and their burst times as shown in Table 2.0. The processes are assumed to arrive at the same time.

Processes	Burst Time (ms)
P1	6
P2	1
P3	2
P4	1
P5	5
P6	4

Table 2.0

Based on the above table, draw a timeline to turn it around and calculate the average turn-around and waiting times for Non Pre-Emptive, First Come, First Served and Shortest Job First scheduling algorithms.

[6 Marks]

- (b) Let us consider a process is running through the *Round Robin* process scheduling policy. If the process consumes the full time slice, the process state is changed from running to ready or blocked state and it is pushed to the end of ready or blocked queue.

What is the main reason for this change?

[4 Marks]

- (c) Briefly explain how a multi-threaded application can be supported by a user-level threads package.

[3 Marks]

- (d) Briefly define the three techniques for performing I/O.

[6 Marks]

- (e) Disk requests come in to the disk driver for cylinders in the order 10, 22, 20, 2, 40, 6 and 38. A seek time takes 3 sec per cylinder move. In all cases, the arm is initially at cylinder 20. Calculate the seek time when the following algorithms are used.

(i) First-come, first served.

(ii) Shortest seek time first.

(iii) Elevator algorithm.

[6 Marks]

3. Critical Section, Deadlock and Process synchronisation (25 Marks)

- (a) What is a critical section problem? Name and briefly explain the three requirements for the solution to critical section problem [5 Marks]
- (b) What is meant by *indefinite postponement* and how does it differ from a *deadlock*? [4 Marks]
- (c) Discuss the consequences of indefinite postponement in each of the following types of systems.
- (i) Batch processing
 - (ii) Timesharing
 - (iii) Real-time [6 Marks]
- (d) Dijkstra's Banker's algorithm has a number of weaknesses that preclude its effective use in real systems. Briefly describe any two of its weaknesses. [4 Marks]
- (e) Suppose the following two processes, P1 and P2 are executed concurrently and share the semaphore variables S and R (each initialized to 1) and the integer variable x (initialized to 0).

<pre> void P1 () { do { semWait(S); semWait(R); x++; semSignal(S); SemSignal(R); } while (1); } </pre>	<pre> void P2 () { do { semWait(R); semWait(S); x--; semSignal(S); SemSignal(R); } while (1); } </pre>
---	---

- (i) Can the concurrent execution of these two processes result in one or both being deadlocked forever? If yes, give an execution sequence in which one or both are deadlocked forever. [3 Marks]
- (ii) Can the concurrent execution of these two processes result in the indefinite postponement of one of them? If yes, give an execution sequence in which one is indefinitely postponed. [3 Marks]

4. Process synchronisation, Mutual Exclusion and semaphore (25 Marks)

- (a) Briefly describe the term *busy waiting*? Is it always wasteful? Write one advantage and disadvantage of busy waiting. [5 Marks]
- (b) Describe the importance and the relevance of Test and Set Lock (TSL) instruction for mutual exclusion. [3 Marks]
- (c) Explain two different ways of implementing Java threads and state if one has an advantage over the other. Why do we call the `start()` method to start the thread instead of calling the `run()` method directly? [4 Marks]
- (d) Describe briefly the producer/consumer problem and show using code how semaphores can be used to solve multiple producers/multiple consumers problem. [6 Marks]

Question continued on the next page

- (e) Consider the following code about the critical-section problem of two-threads:

```

1 System:
2
3 int turn = 1;
4 boolean t1WantsToEnter = false;
5 boolean t2WantsToEnter = false;
6
7 startThreads(); // initialize and launch both threads
8
9 Thread T1:
10
11 void main()
12 {
13     while (!done)
14     {
15         t1WantsToEnter = true;
16
17         while (turn != 1)
18         {
19             while (t2WantsToEnter);
20
21             turn = 1;
22         } // end while
23
24         // critical section code
25
26         t1WantsToEnter = false;
27
28         // code outside critical section
29     } // end outer while
30 } // end Thread T1
31
32 Thread T2:
33
34 void main()
35 {
36     while (! done)
37     {
38         t2WantsToEnter = true;
39
40         while (turn != 2)
41         {
42             while (t1WantsToEnter);
43
44             turn = 2;
45         } // end while
46
47         // critical section code
48
49         t2WantsToEnter = false;
50
51         // code outside critical section
52     } // end outer while
53 } // end Thread T2

```

Does this code enforce the conditions for mutual exclusion? Explain why or why not.

[7 Marks]